

IoT-Based Temperature Monitoring System Technical Report

1. System Operation Overview

This project presents a complete end-to-end IoT monitoring system where an ATmega8A microcontroller acts as a Modbus RTU slave. The microcontroller reads precise temperature data from a J-Type thermocouple, processes it using a Moving Average filter and NIST polynomial calculations, and stores the final value in a Modbus holding register. A Python-based PC application acts as the Modbus Master, fetching this data periodically over a USART serial connection. The Python application then acts as an IoT gateway, transmitting the acquired temperature data to a cloud-based HiveMQ MQTT broker for real-time remote monitoring.

2. USART Communication Between ATmega8A and PC

USART (Universal Synchronous and Asynchronous Receiver-Transmitter) is the hardware protocol used to bridge the ATmega8A and the PC. In this system, asynchronous mode is utilized with a baud rate of 19200 bps. The communication relies on three fundamental lines: RX (Receive), TX (Transmit), and a common GND. Data is framed with a start bit, 8 data bits, and a stop bit. The ATmega8A utilizes hardware interrupts (USART_RXC_vect) to capture incoming bytes from the PC instantly without blocking the main CPU execution, ensuring high-speed and reliable data transfer.

3. Basic Modbus RTU Structure

Modbus RTU is an industry-standard, robust serial communications protocol based on a Master-Slave architecture. The Master (Python PC application) initiates all communication, and the Slave (ATmega8A) strictly responds to requests addressed to its specific ID. A typical Modbus RTU frame consists of:

- **Slave ID (1 Byte):** Identifies the target device on the network (ID: 2 in this project).
- **Function Code (1 Byte):** Instructs the slave on what action to perform (e.g., 0x03 for Read Holding Register).
- **Data Address and Quantity (N Bytes):** Specifies the memory address and how many registers to read/write.
- **CRC (2 Bytes):** A Cyclic Redundancy Check appended at the end of the frame to guarantee data integrity against electrical noise. Frames with mismatched CRCs are silently discarded by the slave.

4. MQTT Publish/Subscribe Architecture

MQTT (Message Queuing Telemetry Transport) is a lightweight, low-bandwidth messaging protocol specifically designed for IoT devices. Unlike traditional client-server models, MQTT uses a Publish/Subscribe architecture. This decouples the data producer from the data consumer. In our system, the Python application is the "Publisher" that sends the temperature data to a specific "Topic" (e.g., iyte/ee446/berke/temperature). Any client (e.g., a web dashboard or a mobile app) that is interested in this data "Subscribes" to the same topic.

5. HiveMQ Cloud Broker Usage

The Broker is the central hub of the MQTT architecture. HiveMQ acts as the cloud-based intermediary that receives all messages from publishers and routes them to the correct subscribers based on the topics. By utilizing the public HiveMQ cloud broker (broker.hivemq.com), the system eliminates the need for complex port forwarding or local server hosting. It allows the temperature data harvested by the ATmega8A to be instantly observable from any internet-connected device worldwide.

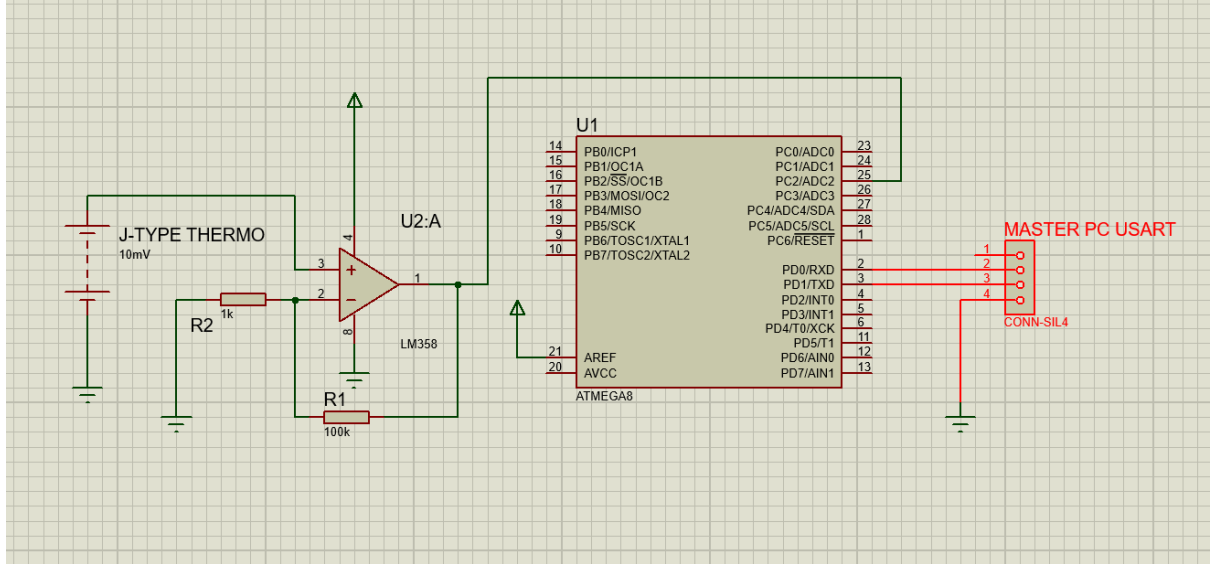
6. Advantages of IoT-based Monitoring Systems

Integrating embedded systems into IoT infrastructures provides several critical industrial advantages:

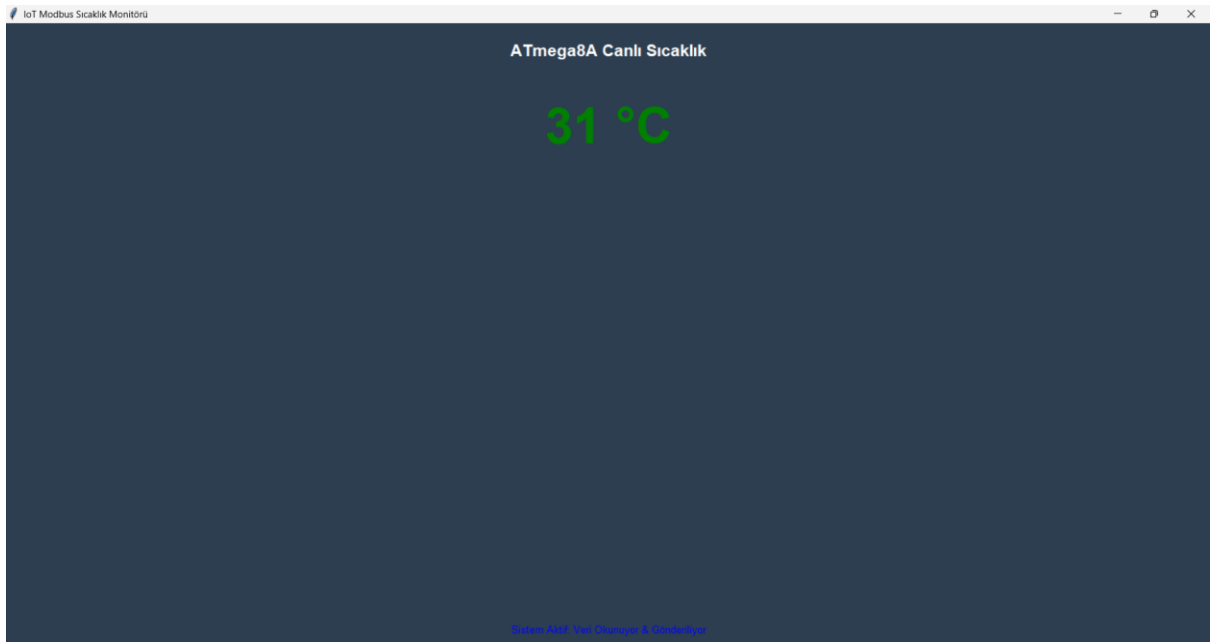
- **Remote Accessibility:** Machine statuses and environmental parameters can be monitored globally in real-time, removing geographic limitations.
- **Scalability:** The Publish/Subscribe model allows thousands of sensors to be added to the network without redesigning the architecture.
- **Predictive Maintenance:** Continuous data logging allows algorithms to detect anomalies (like overheating components) before critical failures occur.
- **Resource Efficiency:** Lightweight protocols like MQTT minimize bandwidth consumption and reduce the processing overhead on edge devices.

7. System Outputs and Documentation

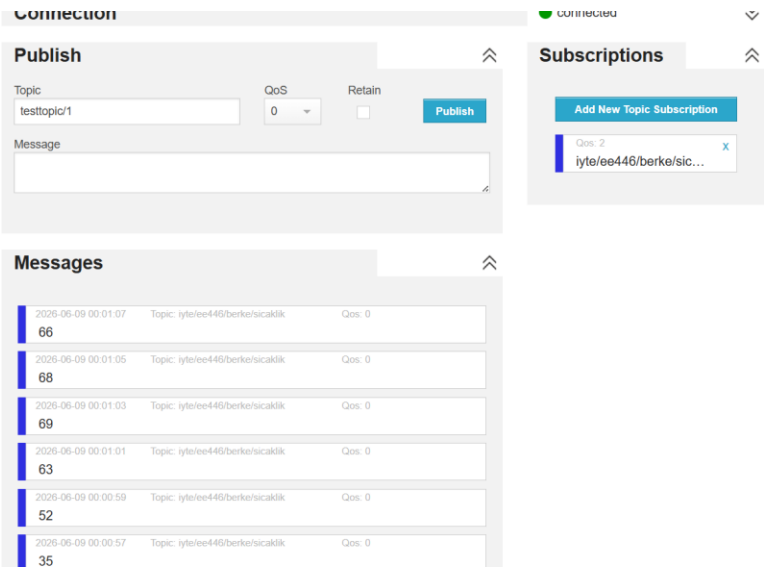
- 7.1 Circuit Diagram



- 7.2 Python PC Application Interface



- 7.3 HiveMQ Web Client Monitoring



2026-06-09 00:01:33	Topic: iyte/ee446/berke/sicaklik	Qos: 0
45		
2026-06-09 00:01:31	Topic: iyte/ee446/berke/sicaklik	Qos: 0
46		
2026-06-09 00:01:28	Topic: iyte/ee446/berke/sicaklik	Qos: 0
47		
2026-06-09 00:01:26	Topic: iyte/ee446/berke/sicaklik	Qos: 0
50		
2026-06-09 00:01:24	Topic: iyte/ee446/berke/sicaklik	Qos: 0
51		
2026-06-09 00:01:22	Topic: iyte/ee446/berke/sicaklik	Qos: 0
54		
2026-06-09 00:01:20	Topic: iyte/ee446/berke/sicaklik	Qos: 0
55		

7.4 MQTT Topic Structure

- Configured Topic: iyte/ee446/berke/sicaklik